

Lecture - 4

❖ Algorithms

- An **algorithm** is a finite sequence of precise, step-by-step instructions for solving a problem or performing a task.
- **Why Algorithms Matter in Programming?**
Before writing code, programmers must design the algorithm. If the algorithm is wrong, the code will be wrong. A good algorithm is:
 - ✓ **Precise:** Each step is clear with no ambiguity
 - ✓ **Ordered:** Steps happen in the right sequence
 - ✓ **Finite:** It eventually reaches an end
 - ✓ **Correct:** It actually solves the problem
- **Characteristics of a Good Algorithm:**
Not all sets of instructions are good algorithms. A quality algorithm must have these five characteristics:
 1. **Finiteness** - The algorithm must have a clear starting point and must eventually end. An infinite loop is not an algorithm.
 2. **Definiteness** - Each step must be clear and unambiguous. "Add some sugar" is not definite — "Add 2 teaspoons" is correct.
 3. **Input** - The algorithm should accept zero or more inputs.
 4. **Output** - It must produce at least one result (output).
 5. **Effectiveness** - Each step must be simple enough to be done manually with just a pen and paper.
- **Three Building Blocks:**
 1. **Sequence**
Instructions executed one after another in a linear path. This is the simplest structure — like reading a book from page 1 to page 10.
Example: Write an algorithm to calculate addition of two numbers
 - Step 1: Start
 - Step 2: Declare integer variables NUM1, NUM2 and SUM.
 - Step 3: Read integer values into NUM1 and NUM2
 - Step 4: Calculate sum of NUM1, NUM2 and stores the result in SUM
 - Step 5: Display the value of SUM
 - Step 6: End

2. Selection (Decision)

Choosing a path based on a condition. Using IF-THEN-ELSE statements to make decisions.

Example: Write an algorithm to check the given integer is even or odd

Step 1: Start

Step 2: Declare an integer variable NUM

Step 3: Read an integer into NUM

Step 4: Check whether $\text{NUM} \% 2 == 0$.

 If yes, go to step 5; otherwise, go to step 6

Step 5: Display "Given number is EVEN" and go to step 7

Step 6: Display "Given number is ODD"

Step 7: End

3. Iteration (Repetition)

Repeating a set of instructions multiple times. Using FOR, WHILE, or REPEAT loops.

Example: Write an algorithm to print 10 natural numbers

Step 1: Start

Step 2: Declare integer variables DES and SOU

Step 3: Read integer value into DES

Step 4: Initialize $\text{SOU} = 1$

Step 5: Check whether $\text{SOU} \leq \text{DES}$. If yes, go to step 6; otherwise, go to step 8

Step 6: Display the value of SOU

Step 7: Increment SOU by 1 and go to step 5

Step 8: End

➤ Algorithm Representation: Pseudocode vs. Flowcharts

There are two main ways to represent algorithms visually:

1. Pseudocode

Describes the logic using code-like instructions without language-specific syntax. It's easier to write than a flowchart but harder to visualize.

- ✓ Uses programming-like keywords (IF, FOR, WHILE)
- ✓ Written in sentence case or uppercase
- ✓ Focuses on logic, not syntax

Examples-1: Write a pseudocode to calculate addition of two numbers

Step 1: START

Step 2: DECLARE NUM1, NUM2 and SUM as INT

Step 3: READ NUM1, NUM2

Step 4: $\text{SUM} \leftarrow \text{NUM1} + \text{NUM2}$

Step 5: PRINT SUM

Step 6: END

Example-2: Write pseudocode to check the given integer is even or odd

Step 1: START
Step 2: DECLARE NUM as INT
Step 3: READ NUM
Step 4: IF NUM%2==0 THEN
 PRINT "Given number is EVEN"
ELSE
 PRINT "Given number is ODD"
END IF
Step 5: END

Example: Write pseudocode to print 10 natural numbers

Step 1: START
Step 2: DECLARE SOU, DES as INT
Step 3: READ DES
Step 4: SET SOU = 1
Step 5: WHILE SOU<=DES THEN
 PRINT SOU
 SET SOU=SOU+1
END WHILE
Step 6: END

2. Flowcharts

A graphical representation using standardized symbols to show the flow of execution.

- ✓ Uses shapes to represent different operations
- ✓ Easy to visualize and understand at a glance
- ✓ Better for complex decision-making
- ✓ Great for planning before coding